

INFORME TÉCNICO

Proyecto Barbería Milla Toledo

Asignatura: Desarrollo FullStack I

Integrantes: Vicente Milla y Vicente Toledo

Proyecto: Barbería Milla Toledo

1. Contenido de la Experiencia 2

El proyecto Barbería Milla Toledo corresponde a una solución backend orientada a la gestión de una barbería. La aplicación permite administrar información relacionada con clientes, barberos, sucursales, servicios, horarios, citas, pagos, inventario, promociones, reseñas y usuarios.

La solución fue desarrollada mediante una arquitectura de microservicios. Cada microservicio se encarga de una responsabilidad específica del sistema y funciona de manera independiente. Cada servicio fue desarrollado utilizando Spring Boot y una estructura compuesta por Controller, Service, Repository y Model.

Los microservicios implementados son:

-horario

-service-auth

-inventario

-clientes

-barberos

-cita

-servicios

-pago

-horario

-promoción

-reseña

-gateway

Diagramas

Diagrama de arquitectura

La arquitectura del proyecto está compuesta por múltiples microservicios independientes. El API Gateway centraliza las solicitudes externas y las redirige hacia el microservicio correspondiente.

El Gateway se encuentra configurado en el puerto 8099. Los microservicios se ejecutan en puertos independientes, lo cual permite separar responsabilidades y mantener una arquitectura distribuida.

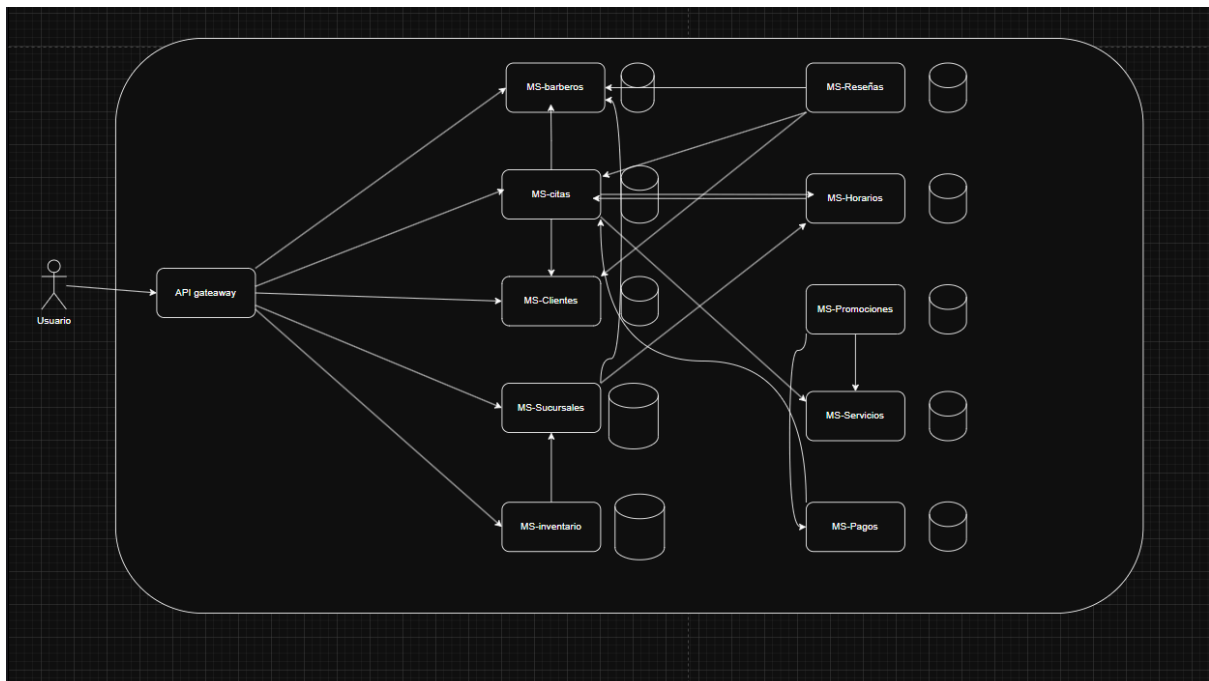
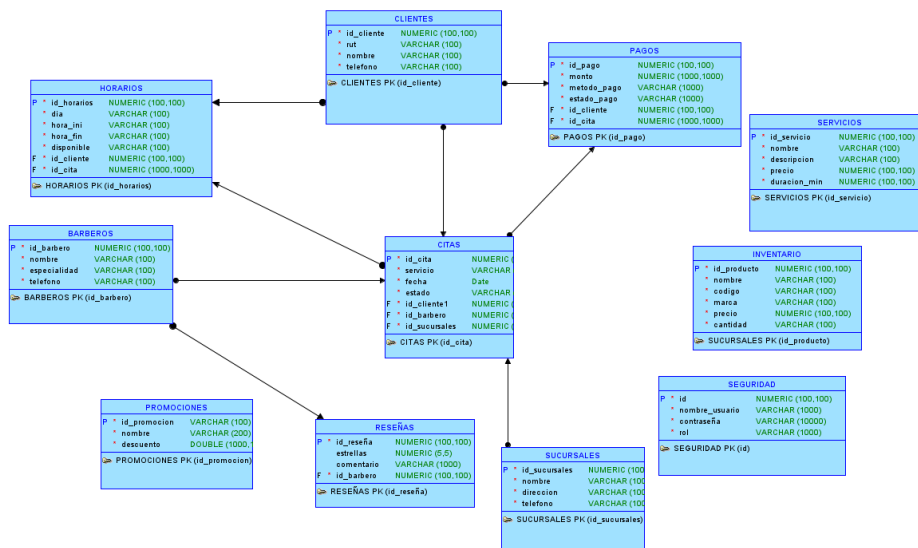


Diagrama de base de datos

El proyecto utiliza bases de datos MySQL para almacenar la información de cada dominio. Se incluyen las principales entidades del proyecto, tales como usuarios, clientes, barberos, citas, servicios, pagos, productos, sucursales, promociones, reseñas y horarios.



3. Servicio de autenticación

El servicio de autenticación corresponde al microservicio `service-auth`, ejecutado en el puerto 8090. Su función es registrar usuarios, validar credenciales e iniciar sesión.

Este servicio permite controlar el acceso a funcionalidades protegidas dentro del sistema.

Autenticación por token

La autenticación se implementa mediante JWT, también conocido como JSON Web Token.

Cuando un usuario inicia sesión, el sistema valida sus credenciales. Si los datos son correctos, se genera un token JWT. Este token se debe enviar en las solicitudes posteriores para demostrar que el usuario está autenticado.

El token se envía mediante el encabezado:

El flujo de autenticación es el siguiente:

1. El usuario se registra mediante la ruta `/auth/regarstrar` .
2. El usuario inicia sesión mediante `/auth/login` .
3. El servicio valida el correo y contraseña.
4. El sistema genera un token JWT.
5. El usuario envía el token en las solicitudes protegidas.

Spring Security

Spring Security se utiliza para configurar la seguridad del microservicio de autenticación.

La configuración permite que las rutas de registro e inicio de sesión sean públicas. Las rutas que requieren seguridad pueden validarse mediante el token JWT recibido en el encabezado de autorización.

Swagger también se encuentra permitido dentro de la configuración de seguridad para poder consultar y probar la documentación de los endpoints.

CORS

CORS permite controlar qué aplicaciones externas pueden realizar solicitudes hacia la API.

Esta configuración es necesaria para permitir que un frontend u otro cliente pueda consumir los endpoints del proyecto sin ser bloqueado por políticas del navegador.

Implementación por capas

-El proyecto utiliza una arquitectura por capas basada en Controller, Service, Repository y Model.

-La capa Controller recibe las solicitudes HTTP de los clientes. En esta capa se definen las rutas REST, tales como GET, POST, PUT y DELETE.

-La capa Service contiene la lógica de negocio. Aquí se validan datos, se controlan reglas del sistema y se coordinan las operaciones necesarias antes de acceder a la base de datos.

-La capa Repository permite realizar operaciones sobre la base de datos mediante Spring Data JPA.

-La capa Model representa las entidades del sistema y sus atributos. Estas entidades se relacionan con las tablas de la base de datos.

Ejemplo por capas

Como ejemplo se utiliza el microservicio barbero, ejecutado en el puerto 8083. El flujo de una petición es el siguiente:

1. El Controller recibe la solicitud HTTP.
2. El Service procesa la lógica de negocio.
3. El Repository realiza la operación en la base de datos.
4. El resultado vuelve al Controller.
5. El Controller responde al cliente mediante HTTP.

Comunicación entre microservicios

La comunicación entre microservicios se realiza mediante solicitudes REST.

El microservicio de citas incorpora WebClient para consumir información remota cuando es necesario. Esta herramienta permite realizar llamadas HTTP hacia otros microservicios y recibir respuestas para continuar el flujo de negocio.

Además, el API Gateway centraliza las rutas y dirige las solicitudes hacia los servicios configurados.

Swagger

Swagger es una herramienta de documentación para APIs REST. Permite visualizar los endpoints disponibles, los parámetros, los cuerpos de solicitud y las respuestas de cada servicio.

La implementación utiliza Springdoc OpenAPI, biblioteca que genera la documentación a partir de los controladores y anotaciones del proyecto.

¿Cómo se implementa?

Swagger se implementa agregando la dependencia en pom.xml del proyecto:

```
<dependency>

<groupId>org.springdoc</groupId>

<artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>

<version>2.7.0</version>

</dependency>
```

Evidencia de Swagger en tres microservicios

Microservicio

Reseña

Ruta:

```
ResenaController.java 1 application.properties pom.xml x ResenaRepository.java 1 ResenaApplication.java 1
pom.xml
2 <project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
23 <scm>
24 <connection/>
25 <developerConnection/>
26 <tag/>
27 <url/>
28 </scm>
29 <properties>
30 <java.version>25</java.version>
31 </properties>
32 <dependencies> Add Spring Boot Starters...
33 <dependency>
34 <groupId>org.springframework.boot</groupId>
35 <artifactId>spring-boot-starter-hateoas</artifactId>
36 </dependency>
37 <dependency>
38 <groupId>org.springframework.boot</groupId>
39 <artifactId>spring-boot-starter-webflux</artifactId>
40 </dependency>
41 <dependency>
42 <groupId>org.springdoc</groupId>
43 <artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>
44 <version>2.7.0</version>
45 </dependency>
46 <dependency>
47 <groupId>org.springframework.boot</groupId>
48 <artifactId>spring-boot-starter-data-jpa</artifactId>
49 </dependency>
```

POST

/api/v1/resenas

Crear una nueva reseña

^

Parameters

Cancel

Reset

No parameters

Request body required

application/json

```
{
  "estrellas": 5,
  "comentario": "Excelente servicio, muy detallista",
  "barberoId": 1
}
```

Execute

Clear

Responses

Curl

```
curl -X 'POST' \
  "http://localhost:8098/api/v1/resenas" \
  -H 'accept: */*' \
  -H 'Content-Type: application/json' \
  -d '{
    "estrellas": 5,
    "comentario": "Excelente servicio, muy detallista",
    "barberoId": 1
  }'
```

Request URL

http://localhost:8098/api/v1/resenas

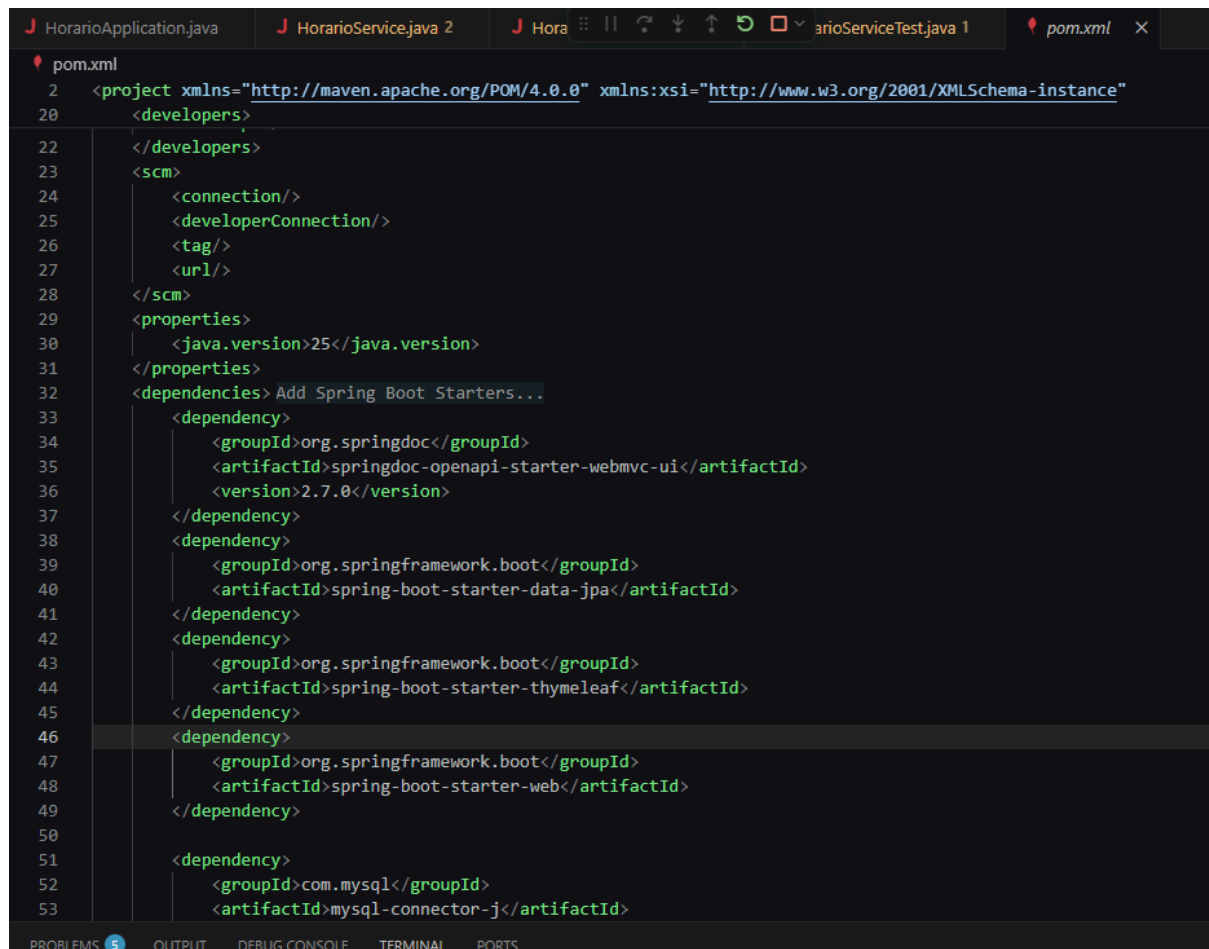
Server response

Code	Details
200	Response body<pre>{ "id": 1, "estrellas": 5, "comentario": "Excelente servicio, muy detallista", "barberoId": 1, "datosBarbero": null }</pre> Download

Response headers

Microservicio

Horario



The screenshot shows an IDE with several tabs open: 'HorarioApplication.java', 'HorarioService.java 2', 'Hora', 'arioServiceTest.java 1', and 'pom.xml'. The 'pom.xml' tab is active, displaying the following XML content:

```
2 <project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
20 <developers>
22 </developers>
23 <scm>
24 <connection/>
25 <developerConnection/>
26 <tag/>
27 <url/>
28 </scm>
29 <properties>
30 <java.version>25</java.version>
31 </properties>
32 <dependencies> Add Spring Boot Starters...
33 <dependency>
34 <groupId>org.springdoc</groupId>
35 <artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>
36 <version>2.7.0</version>
37 </dependency>
38 <dependency>
39 <groupId>org.springframework.boot</groupId>
40 <artifactId>spring-boot-starter-data-jpa</artifactId>
41 </dependency>
42 <dependency>
43 <groupId>org.springframework.boot</groupId>
44 <artifactId>spring-boot-starter-thymeleaf</artifactId>
45 </dependency>
46 <dependency>
47 <groupId>org.springframework.boot</groupId>
48 <artifactId>spring-boot-starter-web</artifactId>
49 </dependency>
50
51 <dependency>
52 <groupId>com.mysql</groupId>
53 <artifactId>mysql-connector-j</artifactId>
```

The bottom of the IDE shows a toolbar with icons for PROBLEMS (5), OUTPUT, DEBUG CONSOLE, TERMINAL, and PORTS.

GET /api/v1/horarios Listar todos los horarios

POST /api/v1/horarios Crear un nuevo horario

Parameters

No parameters

Request body required

application/json

```
{
  "dia": "Lunes",
  "horaInicio": "09:00",
  "horaFin": "18:00",
  "disponible": true
}
```

Execute

Clear

Responses

Curl

```
curl -X 'POST' \
  'http://localhost:8086/api/v1/horarios' \
  -H 'accept: */*' \
  -H 'Content-Type: application/json' \
  -d '{
    "dia": "Lunes",
    "horaInicio": "09:00",
    "horaFin": "18:00",
    "disponible": true
  }'
```

Request URL

http://localhost:8086/api/v1/horarios

Server response

Code

Details

200

Response body

```
{
  "idHorario": 1,
  "dia": "Lunes",
  "horaInicio": "09:00",
  "horaFin": "18:00",
  "disponible": true
}
```

Download

Microservicio Citas

localhost:8085/swagger-ui/index.html#/Citas/guardar

```
{
  "idCliente": 1,
  "idHorario": 1,
  "idProducto": 1,
  "servicio": "Corte de cabello",
  "fecha": "2020-08-30",
  "estado": "pendiente"
}
```

Execute

Clear

Responses

Curl

```
curl -X 'POST' \
  'https://localhost:8085/citas' \
  -H 'accept: */*' \
  -H 'Content-Type: application/json' \
  -d '{
    "idCliente": 1,
    "idHorario": 1,
    "idProducto": 1,
    "servicio": "Corte de cabello",
    "fecha": "2020-08-30",
    "estado": "pendiente"
  }'
```

Request URL

https://localhost:8085/citas

Server response

Code

Details

200

Response body

```
{
  "servicio": "Corte de cabello",
  "fecha": "2020-08-30",
  "estado": "pendiente",
  "idCliente": 1,
  "idHorario": 1,
  "idProducto": 1,
  "idCita": 1
}
```

Download

Response header

```
connection: keep-alive
content-type: application/json
date: Sat, 22 Jun 2020 22:24:48 GMT
keep-alive: timeout=5
transfer-encoding: chunked
```

Responses

Code

Description

Links

200

Cita creada correctamente

No links

Media type

/

```
pom.xml 1 x
cita > pom.xml
2 <project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
5 <parent>
7 <relativePath><!-- lookup parent from repository -->
10 </parent>
11 <groupId>com.citas</groupId>
12 <artifactId>cita</artifactId>
13 <version>0.0.1-SNAPSHOT</version>
14 <name>cita</name>
15 <description/>
16 <url/>
17 <licenses>
18 <license/>
19 </licenses>
20 <developers>
21 <developer/>
22 </developers>
23 <scm>
24 <connection/>
25 <developerConnection/>
26 <tag/>
27 <url/>
28 </scm>
29 <properties>
30 <java.version>21</java.version>
31 </properties>
32 <dependencies> Add Spring Boot Starters...
33 <dependency>
34 <groupId>org.springframework.boot</groupId>
35 <artifactId>spring-boot-starter-data-jpa</artifactId>
36 </dependency>
37 <dependency>
38 <groupId>org.springframework.boot</groupId>
39 <artifactId>spring-boot-starter-web</artifactId>
40 </dependency>
41 <dependency>
42 <groupId>org.springdoc</groupId>
43 <artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>
44 <version>2.8.17</version>
45 </dependency>
46 <dependency>
47 <groupId>org.springframework.boot</groupId>
48 <artifactId>spring-boot-starter-webflux</artifactId>
49 </dependency>
50
51 <dependency>
52 <groupId>org.springframework.boot</groupId>
53 <artifactId>spring-boot-starter-hateoas</artifactId>
54 </dependency>
55
56 <dependency>
57 <groupId>org.springframework.boot</groupId>
58 <artifactId>spring-boot-starter-validation</artifactId>
59 </dependency>
60 <dependency>
61 <groupId>com.mysql</groupId>
62 <artifactId>mysql-connector-j</artifactId>
63 <scope>runtime</scope>
64 </dependency>
65 <dependency>
66 <groupId>org.projectlombok</groupId>
67 <artifactId>lombok</artifactId>
68 <optional>true</optional>
69 </dependency>
70 <dependency>
71 <groupId>org.springframework.boot</groupId>
```

Testing

¿En qué consiste el testing?

El testing consiste en realizar pruebas sobre el código para comprobar que las funcionalidades operen correctamente.

Las pruebas permiten detectar errores antes de ejecutar la aplicación en producción o durante una demostración.

Importancia de testear

Las pruebas unitarias son importantes porque permiten validar reglas de negocio de manera aislada. También ayudan a detectar errores, mejorar la calidad del código y comprobar que una modificación no afecte funcionalidades existentes.

El proyecto utiliza JUnit y Mockito.

JUnit permite crear y ejecutar pruebas. Mockito permite simular dependencias, como repositorios, sin necesidad de utilizar una base de datos real.

Pruebas unitarias y estructura AAA

Las pruebas unitarias se organizan mediante la estructura AAA:

- Arrange: preparación de los datos y mocks necesarios.
- Act: ejecución del método que se desea probar.
- Assert: validación del resultado obtenido.

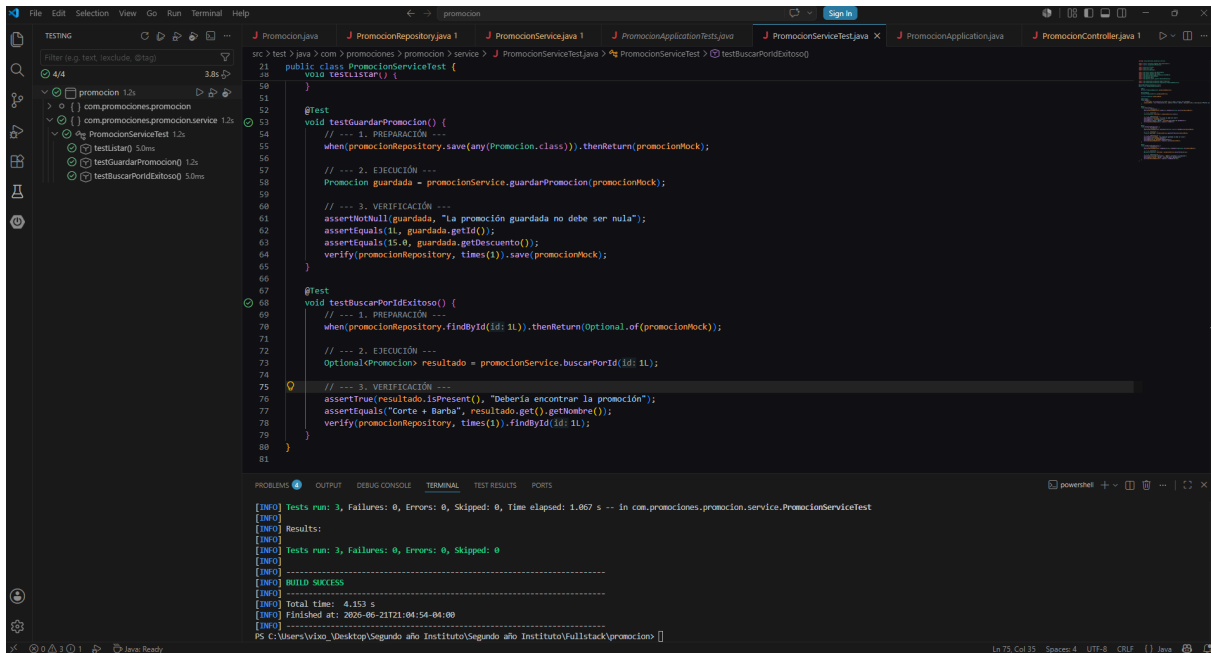
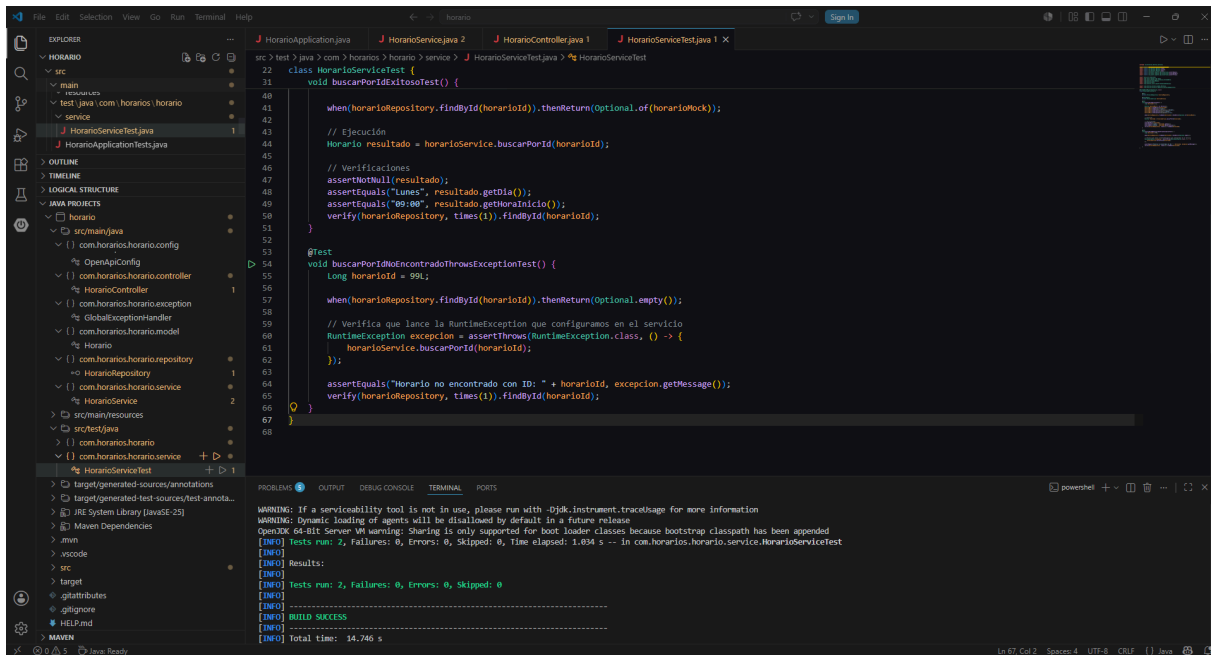
Microservicios testeados

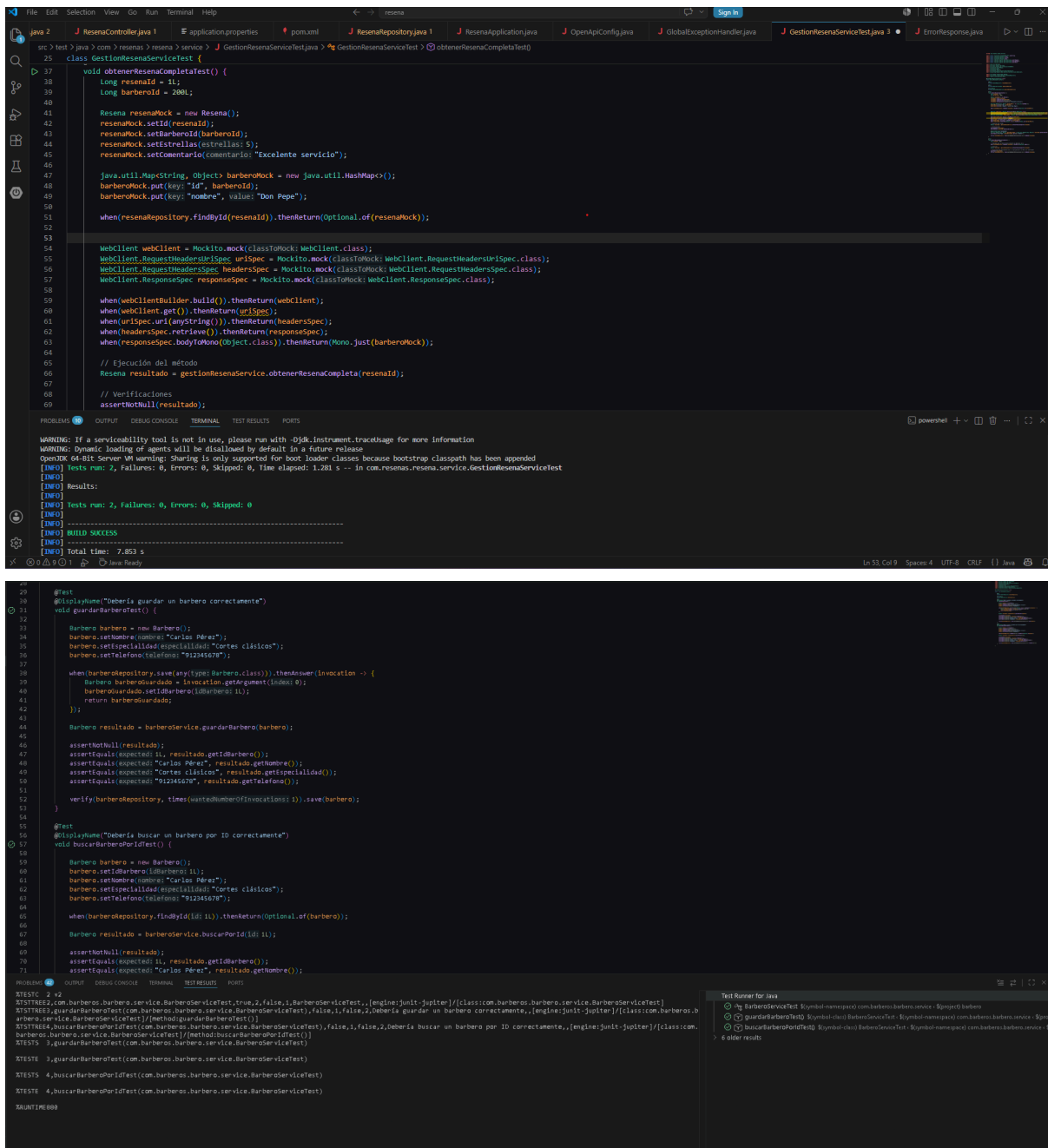
El proyecto incluye pruebas unitarias en 6 microservicios.

Esto permite documentar el 60% de los microservicios testeados, según lo solicitado.

```
Test Runner for Java
  ClientServiceTest $(symbol-namespace) com.clientes.clientes.service - $project clientes
  guardarClienteTest $(symbol-class) ClientServiceTest - $(symbol-namespace) com.clientes.clientes.service - $project clientes
  buscarClientePorIdTest $(symbol-class) ClientServiceTest - $(symbol-namespace) com.clientes.clientes.service - $project clientes
  8 older results
```

```
Test Runner for Java
  [x] InventarioServiceTest $(symbol-namespace) com.inventario.inventario.service, $(project) inventario
  [x] [?] buscarProductoPorIdTest $(symbol-class) InventarioServiceTest, $(symbol-namespace) com.inventario.invent
  [x] [?] guardarProductoTest $(symbol-class) InventarioServiceTest, $(symbol-namespace) com.inventario.inventario
  0 older results
```





Conclusión

El proyecto Barbería Milla Toledo permitió aplicar una arquitectura de microservicios mediante Spring Boot.

La solución incorpora autenticación por token JWT, seguridad con Spring Security, rutas centralizadas mediante API Gateway, documentación Swagger y pruebas unitarias con JUnit y Mockito.

La implementación por capas permite organizar los microservicios mediante Controller, Service, Repository y Model. Además, la comunicación REST entre servicios permitió integrar funcionalidades distribuidas.

